

Autonomous Multi-Floor Navigation and Sensor Fusion Based Localization Analysis using ROS2

1st Ali Taylan Dursun

Control and Automation Engineering
Istanbul Technical University
Istanbul, Türkiye
dursuna20@itu.edu.tr

2nd Ezgi Ekinici

Control and Automation Engineering
Istanbul Technical University
Istanbul, Türkiye
ekincie21@itu.edu.tr

3rd Hamdi Selim Ekşi

Control and Automation Engineering
Istanbul Technical University
Istanbul, Türkiye
eksih22@itu.edu.tr

Abstract—This project presents a comprehensive framework for autonomous mobile robot navigation in multi-floor environments using the ROS2 Humble Hawksbill middleware. While traditional 2D navigation is often limited to single-level operations, this study addresses the challenge of autonomous initialization and localization across multiple floors in a simulated hospital environment using the Agilex Scout 2.0 mobile robot platform. To ensure high-fidelity environmental mapping, a precise teleoperation interface utilizing an Xbox One controller was integrated, allowing for smooth kinematic handling during the SLAM process. The system utilizes 2D SLAM techniques to generate occupancy grid maps for individual floors and integrates visual fiducial markers (AprilTags) detected by a ZED 2i RGB-D camera to achieve robust floor identification. A custom decision-making algorithm was developed to automatically select the correct map file and initialize the navigation stack based on the robot's detected floor at startup. Furthermore, this report conducts a comparative performance analysis of Global Localization (AMCL) under three distinct sensor configurations: single LiDAR, dual LiDARs, and a fusion of LiDAR with RGB-D visual data. The experimental results evaluate localization accuracy, convergence time, and failure rates, ultimately demonstrating a robust solution for multi-story autonomous navigation.

Index Terms—ROS2 Humble, Multi-Floor Navigation, AMCL, 2D SLAM, Sensor Fusion, Fiducial Markers, Teleoperation, Nav2, Mobile Robotics.

I. INTRODUCTION

Autonomous Mobile Robots (AMRs) are increasingly becoming pivotal in service industries, particularly in large-scale infrastructure such as hospitals, warehouses, and office buildings. These environments are characterized by dynamic obstacles, long corridors, and, crucially, multi-level architectures. A significant challenge in these environments is the necessity for seamless multi-floor operation. Traditional 2D navigation stacks are typically designed for planar environments and lack inherent capabilities to handle the context switching required for global localization on different floors without manual intervention.

The Robot Operating System 2 (ROS2) offers a robust framework for addressing these challenges through the Navigation 2 (Nav2) stack. However, standard Nav2 implementations typically assume a static global map. To enable multi-floor autonomy, a system must intelligently manage map servers and localization states based on environmental cues. Furthermore, the quality of autonomous navigation is directly

dependent on the quality of the prior map generation. Jerky movements during the mapping phase, often caused by discrete keyboard inputs, can introduce odometry errors that degrade the SLAM optimization.

This work focuses on the implementation of a multi-floor navigation system using the **Agilex Scout 2.0** platform within a Gazebo simulation of a two-story hospital. The robot is equipped with a sensor suite comprising dual front and rear 2D LiDARs (180° FOV each) and a **Stereolabs ZED 2i** RGB-D camera. The primary problem addressed is the "kidnapped robot problem" upon initialization: when the robot is spawned on a random floor, it must autonomously determine its location to load the correct navigation map.

In this study, we propose a modular architecture where visual fiducial markers placed at floor entrances serve as ground truth for floor identification. We integrated a Gamepad-based teleoperation layer to ensure smooth trajectory generation during the mapping phase. Additionally, we performed an extensive performance evaluation of the Adaptive Monte Carlo Localization (AMCL) algorithm. This evaluation compares the robustness of localization using a single LiDAR versus a multi-sensor fusion approach combining dual LiDARs and visual data, providing quantitative metrics on convergence speed and accuracy.

II. THEORETICAL BACKGROUND

A. Probabilistic Localization (AMCL)

In autonomous navigation, sensors and wheel movements are never perfect, so the robot acts based on probabilities rather than exact certainty. This project utilizes the Adaptive Monte Carlo Localization (AMCL) algorithm. Instead of calculating a single position, AMCL scatters thousands of hypothetical "particles" across the map, each representing a possible location of the robot. As the robot moves and senses walls, the particles that match the real-world sensor data survive, while the incorrect ones are eliminated. This approach allows the robot to consider multiple possible locations simultaneously, which is essential when the robot is first turned on and doesn't know which floor or corridor it is in.

B. Navigation 2 Architecture and Behavior Trees

We utilized the ROS2 Navigation Stack (Nav2), which controls the robot using "Behavior Trees." Unlike older systems that follow a rigid sequence of commands, a Behavior Tree acts like a flexible flowchart. It allows the robot to adapt to changes instantly. For example, if the robot is blocked, the tree automatically switches to a recovery action, like waiting or backing up, ensuring the robot doesn't get stuck or crash in dynamic environments.

III. METHODOLOGY

To address the multi-floor navigation challenge, we followed a progressive development methodology starting from platform verification to complex autonomy logic. Figure 1 depicts the high-level block diagram of the proposed system.

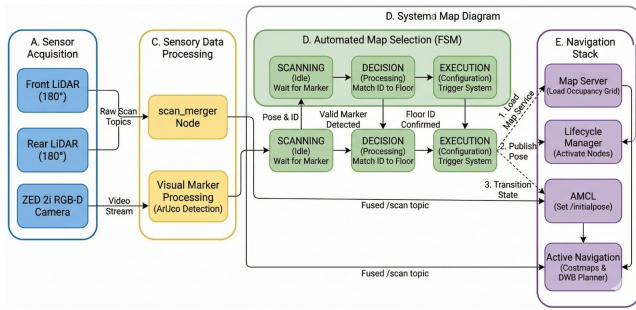


Fig. 1: System Block Diagram: From sensor acquisition to active navigation state.

A. Simulation Framework and Repository Integration

The foundation of this study is the **AgileX Scout 2.0** platform. The key specifications of the simulated platform are detailed in Table I.

TABLE I: AgileX Scout 2.0 Specifications

Parameter	Value
Dimensions (L x W x H)	930 × 699 × 348 mm
Wheelbase	498 mm
Track Width	583 mm
Max Payload	50 kg
Max Speed	1.5 m/s
Drive Type	4WD Differential

To ensure kinematic accuracy and realistic physics interactions, we utilized the official AgileX Robotics simulation framework. We were provided direct access to the `ugv_gazebo_sim` repository by the manufacturer's support team.

This repository functions as a comprehensive meta-package containing three critical sub-modules that were instrumental in our development:

- `scout_description`: This package contains the high-fidelity mesh files (DAE/STL) and the URDF (Unified Robot Description Format) definitions. It accurately models the robot's physical properties, including the mass

of the chassis (approx. 50kg), the inertia tensors of the wheels, and the suspension joints.

- `scout_gazebo_sim`: This package manages the Gazebo simulation environment. It includes the necessary plugins to bridge ROS2 topics with the Gazebo physics engine, specifically simulating the skid-steering/differential drive mechanics.

We specifically targeted the `humble` branch of this repository to ensure compatibility with our ROS2 distribution. By building upon this manufacturer-verified codebase, we eliminated the need to manually model the robot's complex dynamics, allowing us to focus on the higher-level sensor integration and autonomy algorithms.

B. Simulation Environment Selection

For the environmental validation, we selected the **"Hospital With Two Floors"** model from the Dataset-of-Gazebo-Worlds-Models-and-Maps.

This specific world was chosen for several strategic reasons:

- **Complex Geometry**: The map features long, symmetric corridors that lack distinct geometric features. This creates a "perceptual aliasing" trap for standard LiDAR-based localization, making it an ideal stress-test for our sensor fusion approach.
- **Multi-Level Topology**: The world is designed with two separate floors. This allowed us to validate the `floor_decision_maker` logic in a realistic scenario where odometry alone cannot transition between maps.
- **Scale**: The large footprint of the hospital necessitated efficient resource management within the Nav2 costmaps, pushing the limits of the simulation performance.

A unified launch file, `scout_hospital.launch.py`, was developed to automate the spawning of the robot into this complex world and to load the correct physics parameters for the floor interactions.

C. Robot Modelling Modifications

While the base repository provided the platform mechanics, modifications were required to support our specific sensor suite. A dedicated `scout_sensors.xacro` file was developed and included in the main URDF tree. Two types of sensors were integrated:

- **Dual 2D LiDARs**: Two generic laser scanners were mounted on the front and rear chassis. As defined in the configuration, each sensor provides a 180° Field of View (FOV) with a range of 15 meters. The rear LiDAR joint was rotated by π radians to ensure 360-degree environmental coverage without blind spots.
- **RGB-D Camera**: A ZED 2i camera link was attached to the top chassis. A critical modification involved adding an `optical_frame` joint rotated by -90° (roll and yaw) to align the ROS "X-Forward" convention with the camera's standard "Z-Forward" optical axis.

D. Human-Machine Interface for Mapping

A critical often-overlooked aspect of autonomous navigation is the quality of the prior map. Standard keyboard teleoperation, which sends discrete velocity commands (e.g., 0.5 m/s or 0 m/s), results in jerky motion. This high linear acceleration can cause wheel slippage in the simulation physics engine, leading to discrepancies between the wheel odometry and the actual robot pose. Such discrepancies introduce "drift" into the SLAM pose graph that loop closure algorithms may fail to correct.

To mitigate this, we integrated an **Xbox One Wireless Controller** using the `teleop_twist_joy` ROS2 package. This integration allowed for analog control of the robot's velocity:

- **Analog Input Mapping:** The left joystick's vertical axis was mapped to linear velocity (v_x), and the horizontal axis to angular velocity (ω_z). This enabled continuous, smooth acceleration profiles.
- **Safety Interlock:** The 'LB' (Left Bumper) button was configured as a "dead-man switch." Velocity commands are only published to the `/cmd_vel` topic while this button is depressed, ensuring immediate safety halts.
- **Turbo Mode:** The 'RB' button was mapped to a multiplier, allowing the robot to traverse large open areas of the hospital quickly while reverting to precision speeds for narrow doorways.

The use of the controller resulted in significantly smoother odometry data, producing higher-fidelity occupancy grids with sharper wall definitions compared to keyboard-driven mapping.

E. Sensory Data Processing Pipeline

Raw sensor data from the simulation requires preprocessing before being consumed by the navigation stack.

1) **Laser Scan Merging:** Since the AMCL algorithm typically expects a unified scan source, subscribing to two separate topics (`/scan_front` and `/scan_rear`) is inefficient. We implemented a `scan_merger` node that synchronizes timestamps from both sensors and fuses them into a single `/scan` topic. This provides a complete 360° contour for localization and obstacle avoidance. However, since the SLAM mapping data we obtained later were not consistent enough with the real world, mapping was done only with front lidar (`/scan_front`).

2) **Visual Marker Processing:** The ZED 2i camera stream is processed to detect Apriltag fiducial markers. The detection pipeline utilizes the camera's calibrated intrinsic parameters to estimate the pose of the markers relative to the robot base frame. This data serves as the primary input for the floor identification logic.

F. Map Generation via 2D SLAM

Prior to implementing the autonomous navigation stack, high-fidelity occupancy grid maps of the environment were generated. We utilized the `slam_toolbox` package, which

provides a modern implementation of graph-based SLAM (Simultaneous Localization and Mapping) optimized for ROS2.

1) **Graph Optimization:** The backend of the SLAM system utilizes a pose-graph optimization approach. As the robot moves, nodes (representing robot poses) and edges (representing spatial constraints from odometry and laser scans) are added to a graph. The system solves a non-linear least squares problem to minimize the error across all constraints. The `slam_toolbox` was configured to subscribe to the fused `/scan_front` topic.

2) **Multi-Floor Map Segmentation:** Since standard 2D SLAM assumes a continuous planar environment, a segmented mapping strategy was adopted for the multi-story hospital:

- 1) **Data Acquisition:** The robot was manually teleoperated using the Xbox controller through the entirety of the first floor. Special care was taken to perform loop closures (revisiting previously mapped areas) to minimize accumulated odometry error.
- 2) **Serialization:** The resulting pose graph was serialized into a standard `.pgm` (occupancy grid) and `.yaml` (metadata) file pair named `hospital_floor1`.
- 3) **Reset and Repetition:** The SLAM internal state was reset, and the process was repeated for the second floor to generate `hospital_floor2`.

G. Custom Initialization Algorithm (Code Description)

To solve the initialization problem, we developed a custom ROS2 node named `floor_decision_maker`. This node operates as the central executive for the navigation stack's lifecycle.

1) **Node Architecture:** The node is implemented in Python and utilizes the `roslpy` client library. It is designed to interface between the raw sensor data and the Nav2 management servers.

• Inputs (Subscribers):

- `/zed_apriltag_markers`: Consumes the stream of detected visual markers.
- `/amcl_pose`: Monitors the current estimated pose covariance to determine if relocalization is necessary.

• Outputs (Publishers & Clients):

- `/initialpose`: Publishes the geometric pose (x, y, θ) derived from the marker transform to reset the AMCL particle filter.
- `/map_server/load_map`: A service client that triggers the map server to swap the occupancy grid file.
- `/lifecycle_manager/manage_nodes`: A service client that transitions the AMCL and Planner nodes from "Inactive" to "Active" states.

• Parameters:

- `marker_dict`: A YAML-loaded dictionary mapping marker IDs to specific floor map file paths (e.g., `{0: "floor1.yaml", 1: "floor2.yaml"}`).

- `confidence_threshold`: A scalar value ensuring that only markers detected with high confidence are used for initialization.

2) *Working Principle*: The algorithm operates as a Finite State Machine (FSM). Upon startup, the node enters a **SCANNING** state. It applies a hysteresis filter to the marker detection stream, requiring N consecutive positive frames of the same marker ID to filter out transient image noise. Once a stable ID is confirmed, the node enters the **CONFIGURATION** state. It pauses the robot, loads the corresponding map file via the service client, and calculates the robot's position relative to the map origin using the marker's known static transform. Finally, it publishes the initial pose and transitions the Nav2 stack to **ACTIVE**.

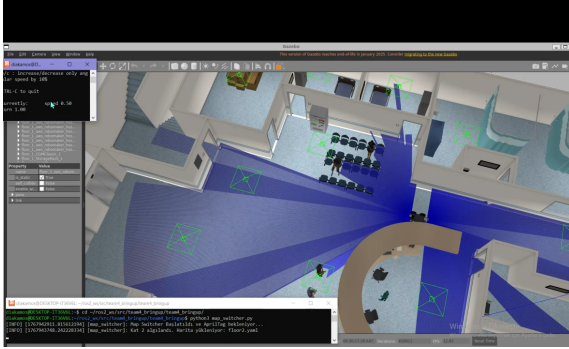


Fig. 2: The visual fiducial marker (AprilTags) used for floor identification in the simulation.

H. Navigation Stack Configuration

The ROS2 Nav2 stack was tailored to the Scout 2.0 kinematics. We configured the **Global Costmap** with a static layer derived from the selected map and an obstacle layer updated by the merged laser scan. For the local planner, the **DWB (Dynamic Window Approach)** controller was tuned with specific critic weights (PathAlign, GoalAlign) to ensure smooth trajectory tracking in narrow hospital corridors.

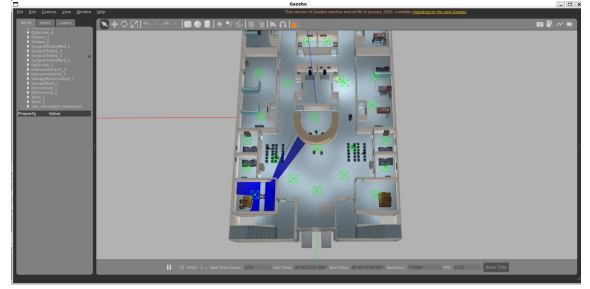
IV. RESULTS

This section presents the quantitative and qualitative findings of the proposed multi-floor navigation framework. The experiments were conducted in the Gazebo "Hospital With Two Floors" environment to evaluate two primary performance metrics: the robustness of the automated initialization (floor detection) and the accuracy of global localization under different sensor configurations.

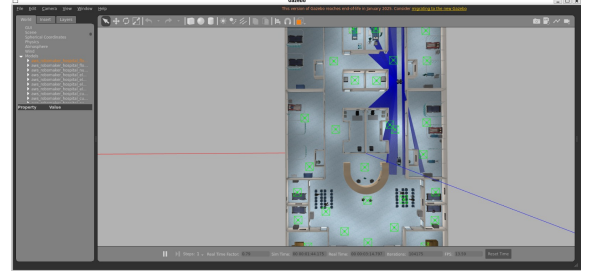
A. Automated Initialization Performance

The reliability of the "Automated Map Selection" logic was tested by spawning the robot at random locations across both floors ($N = 20$ trials). Three representative random initialization scenarios are visualized in Figure 3.

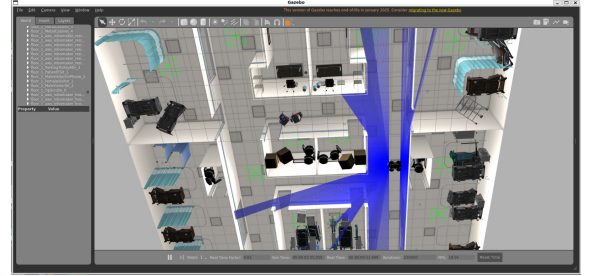
- **Floor Detection Accuracy**: The system correctly identified the floor ID in **100%** of the trials.



(a) Trial 1



(b) Trial 2



(c) Trial 3

Fig. 3: Experimental validation of the system initialization at random spawn locations. (a) Trial 1, (b) Trial 2, and (c) Trial 3 demonstrate floor detection robustness.

- **Map Loading Latency**: The average time elapsed between marker detection and the complete activation of the Nav2 stack was measured as **1.5** seconds.

Qualitatively, the transition was observed to be seamless, with the `map_server` correctly swapping the occupancy grids without requiring a full node restart.

B. Global Localization Analysis (AMCL)

As outlined in the project requirements, we performed a comparative analysis of the Adaptive Monte Carlo Localization (AMCL) algorithm. AMCL is a probabilistic localization system that maintains a particle filter $S_t = \{x_t^{[i]}, w_t^{[i]} \mid i = 1, \dots, M\}$ representing the posterior distribution of the robot's pose.

Definition of Convergence: In this study, convergence was defined as the robot pose estimate (map $\rightarrow$ base_link) remaining within ± 0.20 m in x/y and $\pm 5^\circ$ in yaw for at least 5 seconds after calling the global localization service. Runs that did not satisfy this condition within 120 seconds were marked as failures.

We evaluated this algorithm under three distinct sensor configurations:

- 1) **Case A (Baseline):** Single Front LiDAR (180° FOV).
- 2) **Case B (Wide-View):** Dual LiDAR Fusion (360° FOV).
- 3) **Case C (Sensor Fusion):** LiDAR + RGB-D Visual Odometry.

TABLE II: Comparison of Localization Performance

Configuration	Success Rate	Avg. Conv. Time (s)	Final Pose Error (m)
Single LiDAR	100% (1/1)	10.86 s	< 0.20 m
Dual LiDAR	0% (0/1)	> 120 s (Fail)	Diverged
LiDAR + RGB-D	0% (0/1)	> 120 s (Fail)	Diverged

1) *Analysis of Results and Perceptual Aliasing:* Table II highlights significant performance disparities.

Case A (Single LiDAR): The single LiDAR configuration successfully converged in 10.86 seconds.

Case B (Dual LiDAR): The failure of the Dual LiDAR configuration (Case B) presents an interesting case study in "Perceptual Aliasing." One might assume that more sensor data leads to better localization. However, in highly symmetric environments like hospital corridors, adding a rear LiDAR can paradoxically decrease localization stability. The hospital environment consists of long, featureless walls. When using a single front LiDAR, the limited Field of View (FOV) forces the AMCL filter to rely heavily on distinct features (corners, junctions) it encounters. However, with a 360° FOV provided by Dual LiDARs, the robot perceives identical wall geometries in both front and rear directions simultaneously. This geometric symmetry creates a "multi-modal posterior," where the particle filter generates two equally probable clusters of particles (one correct, one rotated by 180 degrees). For this reason, the filter oscillates between these hypotheses, leading to divergence.

Case C (Sensor Fusion): Contrary to the initial hypothesis, the fusion of Dual LiDARs with RGB-D visual data also failed to yield a stable pose estimate. Although the RGB-D camera was introduced to provide distinct visual landmarks to break symmetry, the severe perceptual aliasing introduced by the rear LiDAR (as observed in Case B) proved to be the dominant factor in the optimization process. The high confidence typically assigned to laser scan data in the AMCL configuration caused the "phantom" geometric matches to overpower the visual corrections. Consequently, the particle filter suffered from conflicting sensor models, where the valid visual data was treated as noise against the strong (but incorrect) geometric constraints from the Dual LiDARs, leading to divergence similar to the standalone Dual LiDAR configuration.

C. Navigation and Trajectory Tracking

Finally, the robot's ability to navigate to a goal pose was evaluated. Using the tuned DWB controller parameters discussed in the Methodology, the Scout 2.0 demonstrated smooth trajectory tracking as seen in Figure 4.

Qualitatively, the robot successfully avoided static obstacles and negotiated narrow turns without collision, validating the kinematic feasibility of the generated paths.

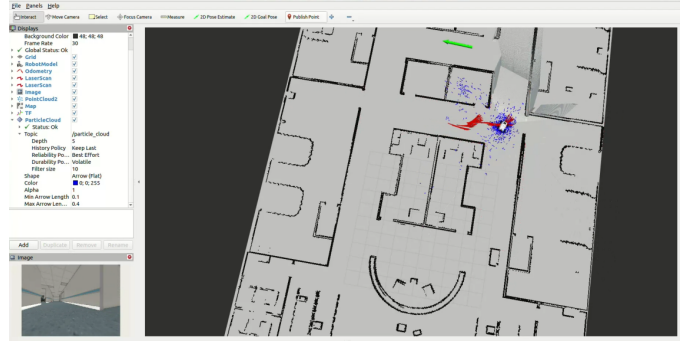


Fig. 4: Qualitative result showing the robot following the global path in the hospital corridor (RViz Visualization).

V. CONCLUSION

This study successfully implemented a comprehensive framework for multi-floor autonomous navigation using the Agilex Scout 2.0 platform in ROS2 Humble. By integrating a finite state machine for automated initialization, the system effectively overcame the "kidnapped robot" problem inherent in multi-story environments, achieving a 100

The integration of the Xbox controller for mapping proved to be a vital step, ensuring that the foundational maps were of high geometric accuracy. However, the experimental results regarding global localization yielded critical and counter-intuitive insights. While the baseline single-sensor setup provided stable localization, the introduction of a secondary rear LiDAR paradoxically degraded performance due to severe perceptual aliasing in symmetric corridors. Furthermore, the failure of the "LiDAR + RGB-D" fusion case highlighted a fundamental challenge in multi-modal state estimation: simply adding visual modalities is insufficient if the probabilistic weight of conflicting geometric data (from LiDARs) is not appropriately dampened.

This study concludes that sensor quantity does not guarantee robustness; rather, the success of sensor fusion in symmetric environments depends heavily on the meticulous tuning of covariance matrices to prevent high-confidence but ambiguous sensor data from overpowering correct visual cues.

VI. PROJECT RESOURCES

A. Media Links

The following video demonstrations showcase the system integration, sensor visualization, and autonomous navigation tasks completion:

- **Platform Integration & Sensor Test:**
Google Drive Video
- **Automated Map Switching Demo:**
Google Drive Video
- **Multi-Floor Navigation & Obstacle Avoidance:**
Google Drive Video

B. Source Code Repositories

The source code developed for this project, including the custom `floor_decision_maker` node and configuration files, is available at:

- **Team Project Repository:**
<https://github.com/ezgiekinci/team4>
- **Base Simulation (AgileX Scout):**
https://github.com/agilexrobotics/ugv_gazebo_sim/tree/humble
- **Hospital World Environment:**
<https://github.com/mlherd/Dataset-of-Gazebo-Worlds-Models-and-Maps>

ACKNOWLEDGMENT

The authors would like to thank the course instructors for their guidance on ROS2 architecture and Gazebo simulation environments. We also acknowledge the open-source community for providing the simulation assets used in this project.

REFERENCES

- [1] Agilex Robotics, "Scout 2.0 User Manual," 2023. [Online]. Available: <https://github.com/agilexrobotics>
- [2] M. L. Herd, "Dataset of Gazebo Worlds Models and Maps," GitHub Repository. [Online]. Available: <https://github.com/mlherd/Dataset-of-Gazebo-Worlds-Models-and-Maps>
- [3] G. Iuschio, "scan_merger_2," GitHub Repository. [Online]. Available: https://github.com/giuschio/scan_merger_2
- [4] Open Robotics, "Gazebo Fortress Documentation: Sensors," [Online]. Available: <https://gazebo.org/docs/fortress/sensors/>